CPS_가상택배터미널_기획서

PLC를 활용한 가상 택배 터미널 물류 분류 체계 기획서

1. 주제 및 프로젝트 개요

1.1 주제

- **프로젝트명**: PLC를 활용한 가상 택배 터미널 물류 분류 체계
- **핵심 개념**: CPS(Cyber-Physical System) 기반으로 가상 물류 공정을 설계하고, PLC 제어, WPF 기반 HMI, MySQL 데이터 저장, Python 분석, OpenCV 보조 검사를 하나의 흐름으로 통합한 스마트 물류 분류 시스템을 구현한다.

1.2 프로젝트 목표

- Factory I/O에서 가상 택배 분류 라인을 구성하고 PLC로 공정을 제어한다.
- PLC와 PC 간 통신을 통해 공정 데이터를 WPF에서 모니터링하고 MySQL에 저장한다.
- 저장된 데이터를 기반으로 지역별 물동량, 처리량, Reject 비율, 알람 패턴을 분석할 수 있는 기반을 만든다.
- OpenCV를 보조 검사 모듈로 설계하여 라벨 이상, 색상 이상, 막힘 및 오분류 확인이 가능하도록 확장성을 확보한다.

1.3 설계 범위

- 분류 목적지: `서울`, `경기`, `부산`, `Reject/Unknown`
- 분류 방식: `RFID 주분류`, `OpenCV 보조 확인`
- 운영 화면: `WPF 운영 HMI + 이벤트 로그 + 이력 조회`
- 저장 대상: `박스 단위 이력`, `분류 이벤트`, `라인 상태`, `알람 기록`
- 실물 연동 범위: `Start/Stop/Reset 버튼`, `Stack Light`, `PLC 기본 I/O`

1.4 설계 기준

- 본 기획서는 수업 가이드와 공식 문서를 기준으로 작성하였다.
- Factory I/O의 RFID Reader는 `Command / Execute Command / Status / Command ID / Read Data` 구조의 단일 트리거 통신 방식을 사용하므로 PLC에서 순차 읽기 시퀀스를 구성하는 방식으로 설계한다.
- XGI CPU는 Local Ethernet에서 `Modbus TCP/IP server` 구성이 가능하므로 Factory I/O 및 WPF가 동일 PLC 메모리맵을 기준으로 데이터를 교환하는 구조로 설계한다.

2. 시스템 전체 구성

2.1 시스템 구성 개요

```
Factory I/O(가상 공정)
    ↓ Modbus TCP Client
PLC(XGT-XGI / XG5000)
    ├─ 실물 I/O(Start/Stop/Reset, Stack Light)
    ├─ WPF HMI와 데이터 교환
    └─ RFID 판독 및 분류 시퀀스 제어
         ↓
WPF(HMI / 로그 / 조회)
         ↓
MySQL(이력 저장)
         ↓
Python(처리량/패턴 분석)
```

```
OpenCV는 카메라 영상 기반 보조 검사 모듈로 별도 동작
```

2.2 구성 요소별 역할

구성 요소	역할
Factory I/O	컨베이어, 센서, RFID Reader, 푸셔, Remover로 구성된 가상 택배 분류 공정 시뮬레이션
PLC(XGI)	공정 시퀀스 제어, RFID 읽기 명령 수행, 지역별 푸셔 제어, 인터록/알람 처리
XG5000	PLC 프로그램 작성, 메모리맵 구성, 통신 파라미터 설정, 디버깅 및 모니터링
WPF	운전 제어, 상태 모니터링, 이벤트 로그, 현재 박스 정보, 지역별 누적 통계 표시
MySQL	분류 이력, 알람 이력, 라인 상태, 기준 정보 저장
Python	시간대별 처리량, 지역별 분류량, Reject 비율, 알람 패턴 분석
OpenCV	라벨/색상 이상 탐지, 오분류 보조 확인, 라인 막힘 감지

2.3 통신 구조

구간	방식	목적
Factory I/O ↔ PLC	Modbus TCP	센서/액추에이터 및 RFID 데이터 교환
WPF ↔ PLC	Modbus TCP	운전 명령, 현재 상태, 집계값, 알람 코드, 현재 박스 정보 조회
WPF ↔ MySQL	DB CRUD	분류 결과 저장, 이력 조회, 기준정보 조회
Python ↔ MySQL	SQL 조회	저장된 이력 데이터 분석
OpenCV ↔ WPF/DB	파일 또는 API 연계 확장	보조 검사 결과 저장 및 표시

2.4 실물 연동 범위

- 실제 PLC 입력: Start 버튼, Stop 버튼, Reset 버튼, Auto/Manual 선택 스위치, 비상정지 접점
- 실제 PLC 출력: 녹색/황색/적색 Stack Light, 부저
- 가상 공정은 Factory I/O에서 시뮬레이션하고, 실물 기기는 운전 제어 및 상태 표시용으로 연결한다.
- 실습 환경에 따라 실제 센서 대신 가상 센서를 우선 사용하고, 최소한 버튼/램프 연동으로 실물 연결 요구사항을 충족한다.

3. 전체 공정 흐름

3.1 기본 흐름

1. `Emitter`가 일정 주기로 박스를 생성한다.
2. 투입부 센서가 박스를 감지하면 PLC는 박스 진입 상태를 기록하고 컨베이어를 구동한다.
3. `RFID Reader 1`이 박스 태그를 읽어 `지역코드`, `무게`, `박스 ID`, `검사 플래그`를 확인한다.
4. PLC는 읽은 값을 내부 메모리에 저장하고 현재 박스 정보를 WPF에 제공한다.
5. 박스가 메인 컨베이어를 따라 이동하는 동안 WPF는 현재 박스 정보와 운전 상태를 실시간 표시한다.
6. 분류 지점 직전 센서가 박스를 감지하면 `RFID Reader 2`가 다시 태그를 읽어 검증한다.
7. PLC는 검증된 지역코드에 따라 `서울`, `경기`, `부산`, `Reject` 중 하나의 목적지 라인을 결정한다.
8. 해당 라인의 푸셔를 일정 시간 구동하여 박스를 목적지 라인으로 분기한다.
9. 출구 센서가 분류 완료를 확인하면 PLC는 완료 펄스를 생성하고 WPF는 이벤트 로그를 기록한다.
10. WPF는 분류 결과를 MySQL에 저장하고, 지역별 누적 수량/누적 무게/실시간 처리량을 갱신한다.
11. Python은 저장된 데이터를 바탕으로 지역별 물동량, 시간대별 처리량, Reject 비율, 알람 패턴을 분석한다.

3.2 RFID 태그 설계

RFID 메모리 인덱스	데이터	설명
`DWORD[0]`	지역코드	`1=서울`, `2=경기`, `3=부산`, `9=Reject/Unknown`
`DWORD[1]`	무게(kg)	정수형 중량 값
`DWORD[2]`	박스 ID	박스 고유 식별 번호
`DWORD[3]`	검사/예외 플래그	정상/검사필요/오류 등 상태 표시

3.3 RFID 운용 규칙

- `RFID Reader 1`은 1차 판독기로 사용한다.
- `RFID Reader 2`는 분류 직전 검증용 판독기로 사용한다.
- Reader는 한 번에 하나의 태그만 읽을 수 있으며, 다중 태그 감지 시 오류 처리한다.
- Factory I/O RFID 태그 메모리는 프로젝트 시작 전 사전 세팅된 상태를 기본 가정으로 한다.
- 초기 구현 시 태그 데이터가 비어 있는 경우에는 `Reader 1에서 write 후 read-back 검증` 방식으로 확장 가능하도록 설계한다.

3.4 예외 처리 정책

상황	처리 방식
RFID 미인식	`Reject` 라인으로 분류하고 알람 로그 기록
다중 태그 감지	`Reject` 처리 후 RFID 오류 알람 발생
지역코드 미등록	`Reject/Unknown` 처리
분류 완료 센서 미감지	타임아웃 알람 발생, 라인 정지
OpenCV 보조 검사 이상	분류는 계속 진행하되 WPF에 경고 표시 및 DB 기록
라인 막힘 감지	컨베이어 정지 및 적색 경광등 점등

4. Factory I/O 공정 구성 요소

4.1 공정 레이아웃 개념

- 1개 투입 라인에서 박스를 생성한다.
- 메인 컨베이어를 따라 RFID 판독 구간과 분류 구간을 지나도록 배치한다.
- 분류 지점에서 4개의 출구 라인(`서울`, `경기`, `부산`, `Reject`)으로 나뉘도록 구성한다.
- 각 출구 라인 끝에는 Remover를 배치해 분류 완료 박스를 씬에서 제거한다.

4.2 공정 구성 요소 목록

구분	구성 요소	수량(예상)	용도
생성	Emitter	1	일정 주기로 박스 생성
이송	Roller/Belt Conveyor	6~8	투입, 메인 이송, 각 목적지 라인 구성
감지	Diffuse Sensor	2	투입 감지, 분류 직전 감지
감지	Retroreflective Sensor	4	각 목적지 라인 분류 완료 확인
판독	RFID Reader	2	1차 판독 및 최종 검증
분기	Pusher	4	서울/경기/부산/Reject 라인 분기
제거	Remover	4	각 라인 종료 지점 박스 제거
제어	Start/Stop 버튼	2	가상 테스트용 운전 스위치
표시	Stack Light	1	운전/대기/알람 상태 표시

4.3 권장 배치 순서

`Emitter → Inlet Sensor → RFID Reader 1 → Main Conveyor → Pre-Sort Sensor → RFID Reader 2 → Pusher 분기 4개 → 각 라인 Confirm Sensor → Remover`

5. 필요한 기물

구분	품목	수량(예상)	용도
PLC	XGT-XGI CPU	1	메인 제어기
PLC	디지털 입력 모듈	1	Start/Stop/Reset/선택 스위치 입력
PLC	디지털 출력 모듈	1	경광등, 부저 등 출력
조작부	Start 버튼	1	공정 시작
조작부	Stop 버튼	1	공정 정지
조작부	Reset 버튼	1	알람 및 카운터 리셋
조작부	Auto/Manual 선택 스위치	1	운전 모드 전환
안전	비상정지 스위치	1	비상 정지
표시	Stack Light(3색)	1	운전 상태 표시
표시	부저	1	알람 경보
PC	개발/운영 PC	1	XG5000, Factory I/O, WPF, DB, Python 실행
네트워크	이더넷 케이블 및 스위치 허브	1식	PLC-PC 통신
소프트웨어	XG5000	1	PLC 로직 작성
소프트웨어	Factory I/O	1	공정 시뮬레이션
소프트웨어	Visual Studio + WPF	1	HMI 개발
소프트웨어	MySQL	1	이력 저장
소프트웨어	Python	1	데이터 분석
보조 검사	USB 또는 IP 카메라	1	OpenCV 영상 입력

5.1 비고

- 실제 RFID 하드웨어는 필수로 두지 않고 Factory I/O의 RFID Reader로 가상 구현한다.
- 실물 연결은 조작부 및 상태 표시 장치 위주로 구성하여 구현 난이도와 안정성을 맞춘다.

6. I/O 리스트

> 아래 주소는 예시 주소이며 실제 슬롯 구성에 따라 변경 가능하다.

6.1 실물 I/O

신호명	방향	구분	주소영역	설명	WPF 표시 여부	DB 저장 여부
Start PB	Input	실물	`%IX0.0.0`	공정 시작 버튼	Y	N
Stop PB	Input	실물	`%IX0.0.1`	공정 정지 버튼	Y	N
Reset PB	Input	실물	`%IX0.0.2`	알람/카운터 리셋	Y	N
Auto/Manual SW	Input	실물	`%IX0.0.3`	운전 모드 선택	Y	N
Emergency Stop	Input	실물	`%IX0.0.4`	비상 정지 입력	Y	Y
Alarm Reset	Input	실물	`%IX0.0.5`	경고 해제	Y	N
Green Lamp	Output	실물	`%QX0.0.0`	운전 중 표시	Y	N
Yellow Lamp	Output	실물	`%QX0.0.1`	대기/일시정지 표시	Y	N
Red Lamp	Output	실물	`%QX0.0.2`	알람 상태 표시	Y	Y
Buzzer	Output	실물	`%QX0.0.3`	알람 경보 출력	Y	Y

6.2 Factory I/O 디지털 I/O

신호명	방향	구분	주소영역	설명	WPF 표시 여부	DB 저장 여부
Inlet Sensor	Input	가상	`%IX1.0.0`	투입부 박스 감지	Y	Y
Pre-Sort Sensor	Input	가상	`%IX1.0.1`	분류 직전 박스 감지	Y	Y
Seoul Confirm Sensor	Input	가상	`%IX1.0.2`	서울 라인 분류 완료 확인	Y	Y
Gyeonggi Confirm Sensor	Input	가상	`%IX1.0.3`	경기 라인 분류 완료 확인	Y	Y
Busan Confirm Sensor	Input	가상	`%IX1.0.4`	부산 라인 분류 완료 확인	Y	Y
Reject Confirm Sensor	Input	가상	`%IX1.0.5`	Reject 라인 분류 완료 확인	Y	Y
Emitter Emit	Output	가상	`%QX1.0.0`	박스 생성 신호	N	N
Main Conveyor Run	Output	가상	`%QX1.0.1`	메인 컨베이어 구동	Y	Y
Seoul Pusher	Output	가상	`%QX1.0.2`	서울 라인 푸셔	Y	Y
Gyeonggi Pusher	Output	가상	`%QX1.0.3`	경기 라인 푸셔	Y	Y
Busan Pusher	Output	가상	`%QX1.0.4`	부산 라인 푸셔	Y	Y
Reject Pusher	Output	가상	`%QX1.0.5`	Reject 라인 푸셔	Y	Y
RFID1 Execute	Output	가상	`%QX1.1.0`	RFID Reader 1 실행	N	N
RFID2 Execute	Output	가상	`%QX1.1.1`	RFID Reader 2 실행	N	N

6.3 Factory I/O RFID 레지스터 I/O

신호명	방향	구분	주소영역	설명	WPF 표시 여부	DB 저장 여부
RFID1 Command	Output	가상	`%MW10`	`0:Tag Check, 1:Serial, 2:Read, 3:Write`	N	N
RFID1 Memory Index	Output	가상	`%MW11`	읽기/쓰기 대상 메모리 인덱스	N	N
RFID1 Write Data	Output	가상	`%MW12`	쓰기 데이터	N	N
RFID1 Command ID	Input	가상	`%MW13`	명령 완료 확인용 카운터	N	N
RFID1 Status	Input	가상	`%MW14`	RFID1 상태 코드	Y	Y
RFID1 Read Data	Input	가상	`%MW15`	RFID1 읽기 데이터	N	Y
RFID2 Command	Output	가상	`%MW20`	RFID2 명령 워드	N	N
RFID2 Memory Index	Output	가상	`%MW21`	RFID2 메모리 인덱스	N	N
RFID2 Write Data	Output	가상	`%MW22`	RFID2 쓰기 데이터	N	N
RFID2 Command ID	Input	가상	`%MW23`	RFID2 명령 완료 확인	N	N
RFID2 Status	Input	가상	`%MW24`	RFID2 상태 코드	Y	Y

신호명	방향	구분	주소영역	설명	WPF 표시 여부	DB 저장 여부
RFID2 Read Data	Input	가상	`%MW25`	RFID2 읽기 데이터	N	Y

6.4 내부 메모리 및 WPF 공유 데이터

신호명	방향	구분	주소영역	설명	WPF 표시 여부	DB 저장 여부
System Run Latch	Internal	내부메모리	`%MB10.0`	시스템 운전 래치	Y	Y
Fault Latch	Internal	내부메모리	`%MB10.1`	시스템 알람 래치	Y	Y
Sort Complete Pulse	Internal	내부메모리	`%MB10.2`	분류 완료 이벤트 펄스	N	Y
Current Box ID	Shared	내부메모리	`%MW100`	현재 박스 ID	Y	Y
Current Region Code	Shared	내부메모리	`%MW101`	현재 박스 지역코드	Y	Y
Current Weight	Shared	내부메모리	`%MW102`	현재 박스 무게	Y	Y
Current Inspect Flag	Shared	내부메모리	`%MW103`	검사/예외 플래그	Y	Y
Last Sort Result	Shared	내부메모리	`%MW104`	최근 분류 결과	Y	Y
Alarm Code	Shared	내부메모리	`%MW105`	현재 알람 코드	Y	Y
Seoul Count	Shared	내부메모리	`%MW110`	서울 누적 수량	Y	Y
Seoul Weight	Shared	내부메모리	`%MW111`	서울 누적 무게	Y	Y
Gyeonggi Count	Shared	내부메모리	`%MW112`	경기 누적 수량	Y	Y
Gyeonggi Weight	Shared	내부메모리	`%MW113`	경기 누적 무게	Y	Y
Busan Count	Shared	내부메모리	`%MW114`	부산 누적 수량	Y	Y
Busan Weight	Shared	내부메모리	`%MW115`	부산 누적 무게	Y	Y
Reject Count	Shared	내부메모리	`%MW116`	Reject 누적 수량	Y	Y
Reject Weight	Shared	내부메모리	`%MW117`	Reject 누적 무게	Y	Y
Throughput Per Min	Shared	내부메모리	`%MW118`	분당 처리량	Y	Y
State Code	Shared	내부메모리	`%MW119`	현재 운전 상태 코드	Y	Y

7. PLC 메모리맵

7.1 메모리 영역 구성

영역	주소 범위(예시)	용도
M Bit	`%MB0.0 ~ %MB9.7`	HMI 명령, 인터록, 상태 비트
M Bit	`%MB10.0 ~ %MB19.7`	운전/알람/분류 완료 플래그
W Word	`%MW10 ~ %MW25`	RFID Reader 제어 및 결과
W Word	`%MW30 ~ %MW39`	판독/검증 중간 데이터
W Word	`%MW100 ~ %MW119`	WPF/DB 공유 영역
T Timer	`%T0 ~ %T9`	푸셔 동작 시간, 센서 타임아웃, 알람 지연

7.2 상태 비트 정의

주소	명칭	설명
`%MB10.0`	RUN_LATCH	시스템 운전 유지
`%MB10.1`	FAULT_LATCH	알람 상태 유지
`%MB10.2`	SORT_COMPLETE	분류 완료 펄스
`%MB10.3`	RFID1_REQ	RFID1 읽기 요청
`%MB10.4`	RFID1_DONE	RFID1 읽기 완료
`%MB10.5`	RFID2_REQ	RFID2 읽기 요청
`%MB10.6`	RFID2_DONE	RFID2 읽기 완료
`%MB11.0`	SORT_TO_SEOUL	서울 분류 비트
`%MB11.1`	SORT_TO_GYEONGGI	경기 분류 비트
`%MB11.2`	SORT_TO_BUSAN	부산 분류 비트
`%MB11.3`	SORT_TO_REJECT	Reject 분류 비트
`%MB11.4`	SORT_VALID	최종 분류 판단 완료
`%MB11.5`	JAM_ALARM	라인 막힘 알람
`%MB11.6`	RFID_ALARM	RFID 관련 알람
`%MB11.7`	CV_WARNING	OpenCV 보조 검사 경고

7.3 워드 데이터 정의

주소	명칭	설명
`%MW30`	VALIDATED_REGION	최종 검증된 지역코드
`%MW31`	VALIDATED_WEIGHT	최종 검증된 무게
`%MW32`	VALIDATED_BOX_ID	최종 검증된 박스 ID
`%MW33`	VALIDATED_FLAG	최종 검사 플래그
`%MW34`	LAST_RFID_ERROR	최근 RFID 오류 코드
`%MW35`	LAST_SORT_LINE	최근 분류 라인
`%MW105`	ALARM_CODE	현재 알람 코드
`%MW119`	STATE_CODE	`0:Idle, 1:Running, 2:Reading, 3:Sorting, 4:Alarm`

7.4 알람 코드 정의

코드	내용
`0`	정상
`1`	RFID1 No Tag
`2`	RFID Too Many Tags
`3`	지역코드 미등록
`4`	분류 완료 센서 타임아웃
`5`	라인 막힘 감지
`6`	비상정지 입력
`7`	OpenCV 경고

7.5 핵심 LD 레더 예시

1) 운전 래치

```
   ┌──┤%IX0.0.0├──┐
──┤               ├──┤/%IX0.0.1├──┤/%IX0.0.4├──────────────────────(S)── %MB10.0
   └──┤%MB10.0├──┘
```

```
──┤%IX0.0.1├──────────────────────────────────────────────────────(R)── %MB10.0
──┤%IX0.0.4├──────────────────────────────────────────────────────(R)── %MB10.0
```

2) 지역별 분류 비트 결정

```
──┤%MB11.4├──┤EQ %MW30,1├──────────────────────────────────────(S)── %MB11.0
──┤%MB11.4├──┤EQ %MW30,2├──────────────────────────────────────(S)── %MB11.1
──┤%MB11.4├──┤EQ %MW30,3├──────────────────────────────────────(S)── %MB11.2
──┤%MB11.4├──┤EQ %MW30,9├──────────────────────────────────────(S)── %MB11.3
```

3) 푸셔 동작 타이머 예시

```
   ┌──┤%MB11.0├──┐
   ├──┤%MB11.1├──┤
──┤              ├──────────────────────────────────────┤/%T0.Q├──[TON T0, PT:=T#1s]
   ├──┤%MB11.2├──┤
   └──┤%MB11.3├──┘
```

8. DB 설계

8.1 DB 설계 목표

- 박스별 이력 추적이 가능해야 한다.
- 분류 완료 이벤트와 알람 이벤트를 분리 저장해야 한다.
- WPF에서 실시간 조회와 과거 이력 조회가 가능해야 한다.
- Python 분석에 필요한 최소 필드를 미리 확보해야 한다.

8.2 테이블 구성

테이블명	목적
`parcel`	박스 기본 정보와 최종 처리 결과 저장
`sort_event`	판독, 검증, 분류 완료 등 이벤트 로그 저장
`line_status_log`	일정 주기별 라인 상태 스냅샷 저장
`alarm_history`	알람 발생/해제 이력 저장
`region_rule`	지역코드와 목적지 라인 기준 정보 저장

8.3 테이블 상세 설계

8.3.1 `parcel`

필드명	타입	키	설명
`parcel_id`	VARCHAR(30)	PK	박스 ID
`created_at`	DATETIME		최초 인식 시각
`region_code`	TINYINT	FK	지역코드
`region_name`	VARCHAR(20)		지역명
`weight_kg`	INT		무게
`inspect_flag`	TINYINT		검사 플래그
`current_status`	VARCHAR(20)		현재 상태
`final_result`	VARCHAR(20)		최종 분류 결과

필드명	타입	키	설명
`sorted_at`	DATETIME		분류 완료 시각
`alarm_code`	INT		마지막 알람 코드

8.3.2 `sort_event`

필드명	타입	키	설명
`event_id`	BIGINT	PK	이벤트 번호
`parcel_id`	VARCHAR(30)	FK	박스 ID
`event_type`	VARCHAR(30)		`READ1`, `READ2`, `SORT_COMPLETE`, `REJECT` 등
`event_time`	DATETIME		이벤트 시각
`line_name`	VARCHAR(20)		목적지 라인
`region_code`	TINYINT		지역코드
`weight_kg`	INT		무게
`alarm_code`	INT		관련 알람 코드
`message`	VARCHAR(200)		WPF 이벤트 로그용 메시지

8.3.3 `line_status_log`

필드명	타입	키	설명
`log_id`	BIGINT	PK	상태 로그 번호
`recorded_at`	DATETIME		기록 시각
`run_state`	VARCHAR(20)		Idle/Running/Alarm
`conveyor_run`	TINYINT		컨베이어 동작 여부
`box_in_process`	INT		현재 처리 중 박스 수
`seoul_count`	INT		서울 누적 수량
`gyeonggi_count`	INT		경기 누적 수량
`busan_count`	INT		부산 누적 수량
`reject_count`	INT		Reject 누적 수량
`jam_alarm`	TINYINT		막힘 알람 여부
`rfid_alarm`	TINYINT		RFID 알람 여부

8.3.4 `alarm_history`

필드명	타입	키	설명
`alarm_id`	BIGINT	PK	알람 번호
`occurred_at`	DATETIME		발생 시각
`cleared_at`	DATETIME		해제 시각
`alarm_code`	INT		알람 코드
`alarm_name`	VARCHAR(50)		알람명
`severity`	VARCHAR(10)		경고/중요
`parcel_id`	VARCHAR(30)	FK	관련 박스 ID
`message`	VARCHAR(200)		상세 메시지
`active_flag`	TINYINT		활성 여부

8.3.5 `region_rule`

필드명	타입	키	설명
`region_code`	TINYINT	PK	지역코드
`region_name`	VARCHAR(20)		지역명
`output_line`	VARCHAR(20)		목적지 라인명
`pusher_address`	VARCHAR(20)		PLC 출력 주소
`active_flag`	TINYINT		사용 여부
`note`	VARCHAR(100)		비고

8.4 테이블 관계

- `region_rule 1 : N parcel`
- `parcel 1 : N sort_event`
- `parcel 1 : N alarm_history`
- `line_status_log`는 라인 운영 상태 스냅샷 테이블로 독립 관리한다.

8.5 CRUD 활용 방안

- **Create**: 분류 완료 시 `parcel`, `sort_event`, `line_status_log` 저장
- **Read**: WPF에서 박스 검색, 기간별 통계, 알람 이력 조회
- **Update**: 기준 지역코드 활성화 여부 변경, 알람 해제 시간 기록
- **Delete**: 테스트 데이터 정리 시 관리자 권한으로만 수행

9. WPF 화면 설계

9.1 화면 구성 목표

- 작업자가 공정을 한 화면에서 파악할 수 있어야 한다.
- 현재 박스 정보, 분류 결과, 누적 통계, 알람 상태를 동시에 볼 수 있어야 한다.
- 운영 제어와 이력 조회를 분리하여 사용성을 높인다.

9.2 화면 구성

화면	주요 기능
메인 운영 화면	시작/정지/리셋, 운전 상태, 현재 박스 정보, 최근 이벤트, 알람 상태
이벤트 로그/이력 조회	기간별 분류 이력, 박스 ID 검색, 알람 검색
통계 요약 패널	지역별 누적 수량, 누적 무게, Reject 비율, 분당 처리량

9.3 메인 운영 화면 와이어프레임

```
+-------------------------------------------------------------------------------
+
| [시작] [정지] [리셋] [자동/수동]     현재상태: Running     알람코드: 0            |
+------------------------+--------------------------------+------------------------
+
| 현재 박스 정보          | 공정 상태 요약                  | 알람/경고              |
| - 박스 ID               | - 컨베이어 동작                 | - RFID 오류            |
| - 지역코드              | - RFID1 판독 완료               | - 라인 막힘            |
| - 무게                  | - RFID2 검증 완료               | - OpenCV 경고          |
| - 검사 플래그           | - 현재 분류 대상 라인           |                        |
+------------------------+--------------------------------+------------------------
+
```

```
| 지역별 누적 통계                                                                  |
| 서울: 수량 / 무게   경기: 수량 / 무게   부산: 수량 / 무게   Reject: 수량 / 무게 |
+-----------------------------------------------------------------------------------
+
| 최근 이벤트 로그                                                                   |
| 12:03:10  서울행 / 52kg / 분류 완료                                              |
| 12:03:16  경기행 / 31kg / 분류 완료                                              |
| 12:03:20  RFID 오류 / Reject 처리                                                 |
+-----------------------------------------------------------------------------------
+
```

9.4 화면별 데이터 항목

화면 요소	PLC/DB 연동 데이터
운전 상태	`%MB10.0`, `%MW119`, 실물 버튼 상태
현재 박스 정보	`%MW100 ~ %MW103`
최근 분류 결과	`%MW104`, `sort_event`
알람 상태	`%MB10.1`, `%MW105`, `alarm_history`
지역별 누적 통계	`%MW110 ~ %MW118`, `line_status_log`
이력 조회	`parcel`, `sort_event`, `alarm_history`

9.5 WPF 동작 정책

- PLC 상태는 주기적으로 Polling하여 갱신한다.
- `Sort Complete Pulse` 감지 시 이벤트 로그 1건을 생성한다.
- 알람 발생 시 상단 상태 영역과 알람 패널을 동시에 갱신한다.
- DB 저장 실패 시 HMI에 통신 경고를 표시하되 PLC 제어는 유지한다.

10. Python/OpenCV 활용 계획

10.1 Python 데이터 분석 계획

분석 항목	설명
지역별 처리량	지역별 누적 분류 수량 및 비율 분석
시간대별 분류량	시간대별 피크 구간 확인
평균 중량 분석	지역별 평균 무게 및 편차 확인
Reject 비율	전체 대비 Reject 비율 분석
알람 패턴 분석	알람 발생 시간대, 유형, 빈도 분석

10.2 Python 시각화 예시

- 시간대별 분류량 막대그래프
- 지역별 누적 물동량 파이차트
- 일자별 Reject 비율 추이선
- 알람 유형별 발생 빈도 그래프

10.3 OpenCV 보조 검사 계획

기능	설명
라벨/색상 이상 확인	박스 라벨 또는 색상 정보가 기준과 맞는지 확인

기능	설명
오분류 보조 확인	예상 목적지와 실제 이동 라인이 불일치하는지 확인
막힘 감지	특정 구간에 박스가 장시간 정체되는지 감지
경고 연동	OpenCV 이상 감지 시 WPF 경고 표시 및 DB 기록

10.4 OpenCV 적용 원칙

- OpenCV는 보조 검사 모듈이며 주 분류 판단에는 직접 관여하지 않는다.
- OpenCV 미동작 시에도 PLC의 RFID 기반 분류 공정은 정상 유지된다.
- OpenCV 결과는 `경고` 성격으로 저장하고, 공정 정지는 PLC 알람 조건에 따라 별도 판단한다.

참고 기준

- 수업 가이드: `CPS_프로젝트_가이드.pdf`
- Factory I/O 공식 문서: Manual, Modbus TCP/IP Client, RFID Reader / Sensors, Emitter, Remover
- LS Electric 공식 문서: XGI CPU Manual